
In Search of an Understandable Consensus Protocol (Raft)

Diego Ongaro and John Ousterhout — Stanford University

Section 1: In Search of an Understandable Consensus Algorithm

Raft is a consensus algorithm for managing a replicated log. It produces a result equivalent to (multi-)Paxos, and it is as efficient as Paxos, but its structure is different from Paxos; this makes Raft more understandable than Paxos and also provides a better foundation for building practical systems.

To enhance understandability, Raft decomposes consensus into relatively independent subproblems, including leader election, log replication, safety, and membership changes.

Section 5.2: Leader Election & Quorums

Raft uses a heartbeat mechanism to trigger leader election. When servers start up, they begin as followers. A server remains in follower state as long as it receives valid RPCs from a leader or candidate.

To begin an election, a follower increments its current term and transitions to candidate state. It then votes for itself and issues RequestVote RPCs in parallel to each of the other servers in the cluster.

A candidate wins an election if it receives votes from a majority of the servers in the full cluster for the same term: $\text{Quorum} = \text{floor}(N/2) + 1$.

Section 5.4: Safety & Leader Completeness Invariant

Raft guarantees the Leader Completeness Property: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms.

A voter denies its vote if the candidate's log is less up-to-date than the voter's own log: RequestVote RPC includes candidate log length and term.